Python Activity-Based Learning

1.

Attendance Recovery Computation: Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

Sol:

Problem statement

Calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach a required attendance percentage.

Objectives

- Calculate attendance percentage.
- Use a loop to find the minimum future classes needed.
- Display a clear eligibility result.

Assumptions

- Classes held and classes attended are non-negative integers.
- Classes attended cannot exceed classes held.
- The required percentage is between 0 and 100.

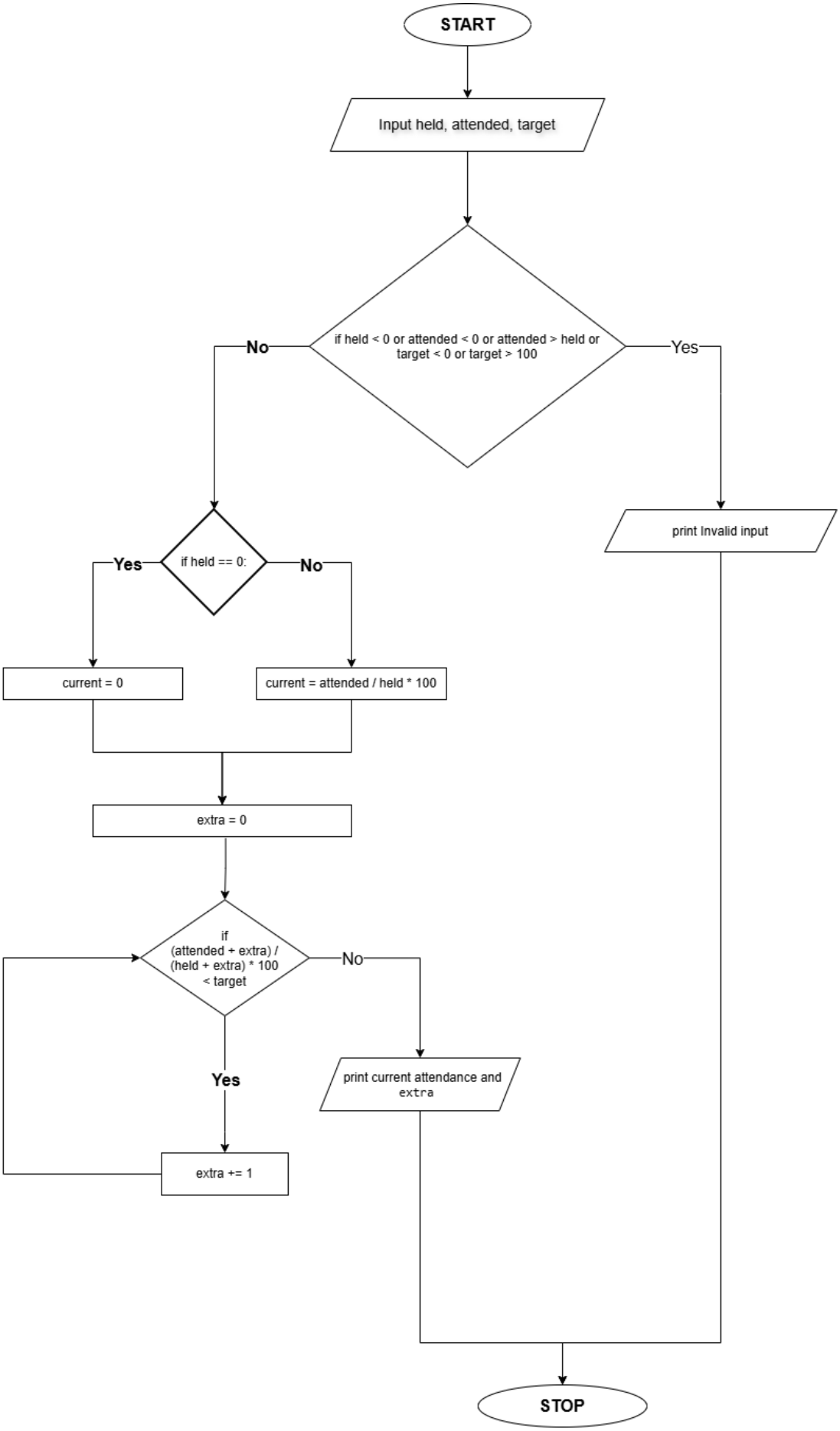
Algorithm

1. Read classes held, classes attended, and required percentage.
2. Validate the inputs.
3. Calculate current attendance percentage.
4. If the target is already reached, print 0 additional classes.
5. Otherwise, repeatedly add one attended class and one held class until the target is reached.
6. Print the percentage and additional classes needed.

Pseudocode

1. START
2. Read Input held, attended, target
3. if held < 0 OR attended < 0 OR attended > held OR target < 0 OR target > 100:
   print Invalid input
4. else:
   if held == 0:
   current = 0
   else:
   current = attended / held * 100
5. Set extra = 0
6. Now, while (attended + extra) / (held + extra) * 100 < target:
   extra+=1
7. print percentage, extra
8. STOP

Flowchart


START
Input held, attended, target
if held < 0 or attended < 0 or attended > held or target < 0 or target > 100
No
Yes
print Invalid input
if held == 0:
Yes
No
current = 0
current = attended / held * 100
extra = 0
if (attended + extra) / (held + extra) * 100 < target
No
Yes
print current attendance and extra
extra += 1
STOP

Source Code

```
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
   print("Invalid input.")
else:
    if held == 0:
      current = 0
    else:
      current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
   extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)
```

Test Cases

- Input: 40; 32; 75
  - Expected result: Current attendance: 80.0 %; Additional classes to attend: 0
- Input: 20; 12; 75
  - Expected result: Current attendance: 60.0 %; Additional classes to attend: 12 •
- Input: 10; 10; 75
  - Expected result: Current attendance: 100.0 %; Additional classes to attend: 0

Output

```python
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
    print("Invalid input.")
else:
    if held == 0:
        current = 0
    else:
        current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
    extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)
```

un 1 ×

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter total classes held: 20
Enter classes attended: 12
Enter required attendance percentage: 75
Current attendance: 60.00%
Additional classes to attend: 12

Process finished with exit code 0
```

```python
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
    print("Invalid input.")
else:
     if held == 0:
        current = 0
     else:
        current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
    extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)

```

Run 1 ×

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter total classes held: 40
Enter classes attended: 32
Enter required attendance percentage: 75
Current attendance: 80.00%
Additional classes to attend: 0

Process finished with exit code 0
```

```python
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
    print("Invalid input.")
else:
    if held == 0:
        current = 0
    else:
        current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
    extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)
```

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter total classes held: 10
Enter classes attended: 10
Enter required attendance percentage: 75
Current attendance: 100.00%
Additional classes to attend: 0

Process finished with exit code 0
```

Conclusion

The program calculates attendance and uses repetition to find the minimum number of future classes required to reach the target

2.

Food Waste Monitoring System: Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

Sol:

Problem statement

Compare the number of meals prepared, meals served, and leftovers over several days; calculate waste percentages and identify days that exceed an acceptable limit.

Objectives

- Calculate leftover meals.
- Calculate waste percentage for each day.
- Identify days above the supplied waste limit.

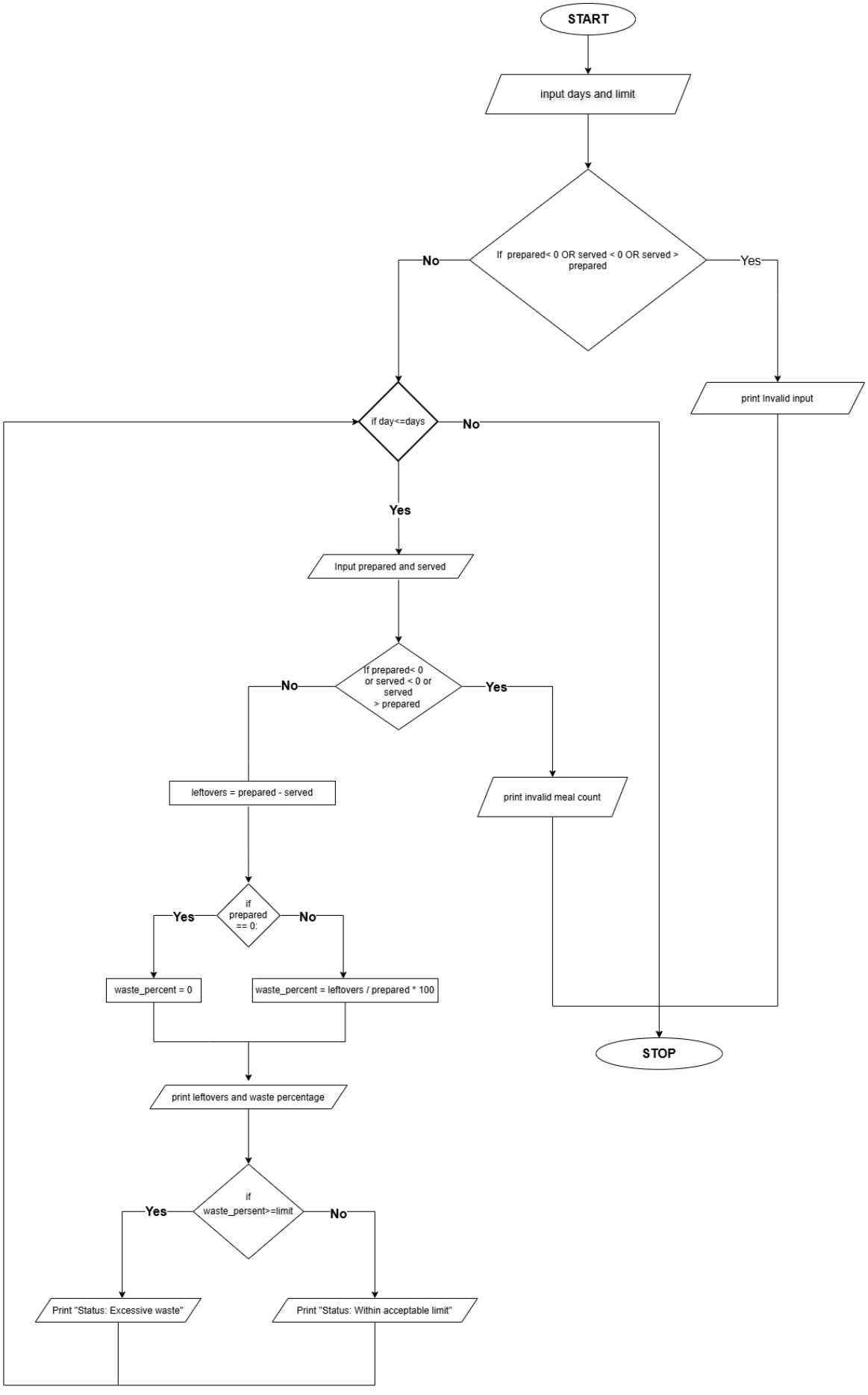
Assumptions

- Prepared and served meals are non-negative integers.
- Meals served cannot exceed meals prepared.
- Waste percentage is leftovers divided by meals prepared, multiplied by 100.

Algorithm

1. Read the number of days and acceptable waste percentage.
2. For each day, read meals prepared and served.
3. Validate the values. 4
4. Calculate leftovers and waste percentage.
5. Classify the day as within limit or excessive waste.
6. Display each day's result.

Flowchart


START
input days and limit
If prepared< 0 OR served < 0 OR served > prepared
No
Yes
print Invalid input
if day<=days
No
Yes
Input prepared and served
If prepared< 0 or served < 0 or served > prepared
No
Yes
print invalid meal count
leftovers = prepared - served
if prepared == 0:
Yes
No
waste_percent = 0
waste_percent = leftovers / prepared * 100
STOP
print leftovers and waste percentage
if waste_persent>=limit
Yes
No
Print "Status: Excessive waste"
Print "Status: Within acceptable limit"

Pseudocode

```
START
INPUT days, limit
IF days <= 0 OR limit < 0 OR limit > 100
  PRINT "Invalid input"
ELSE
  FOR each day
    INPUT prepared, served
    IF prepared < 0 OR served < 0 OR served > prepared
      PRINT "Invalid meal counts"
    ELSE
      leftovers = prepared - served
      IF prepared = 0
        Waste_persent= 0
      ELSE
        Waste_persent= leftovers / prepared * 100
      PRINT leftovers, waste_persent
      IF waste_persent> limit
        PRINT "Excessive waste"
      ELSE
        PRINT "Within acceptable limit"
STOP
```

Source Code

```
days = int(input("Enter number of days: "))
limit = float(input("Enter acceptable waste percentage: "))

if days <= 0 or limit < 0 or limit > 100:
  print("Invalid input.")
else:
```

```
for day in range(1, days + 1):
    prepared = int(input("Meals prepared on day " + str(day) + ": "))
    served = int(input("Meals served on day " + str(day) + ": "))

    if prepared < 0 or served < 0 or served > prepared:
        print("Invalid meal counts.")
    else:
        leftovers = prepared - served

        if prepared == 0:
            waste_percent = 0
        else:
            waste_percent = leftovers / prepared * 100

        print("Leftover meals:", leftovers)
        print("Waste percentage:", round(waste_percent, 2), "%")

        if waste_percent > limit:
            print("Status: Excessive waste")
        else:
            print("Status: Within acceptable limit")
```

Test Cases

1. Enter number of days: 1
   Enter acceptable waste percentage: 20
   Meals prepared on day 1: 100
   Meals served on day 1: 90

   Output:

   Leftover meals: 10

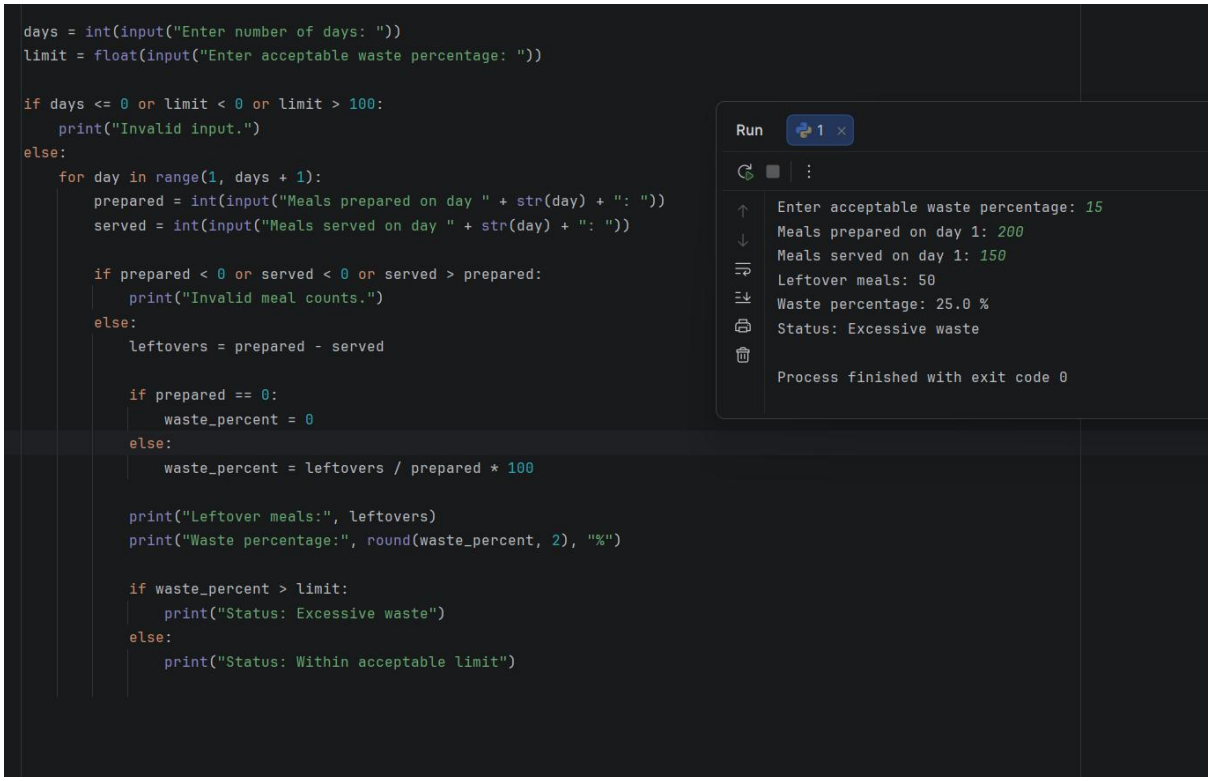
   Waste percentage: 10.0 %

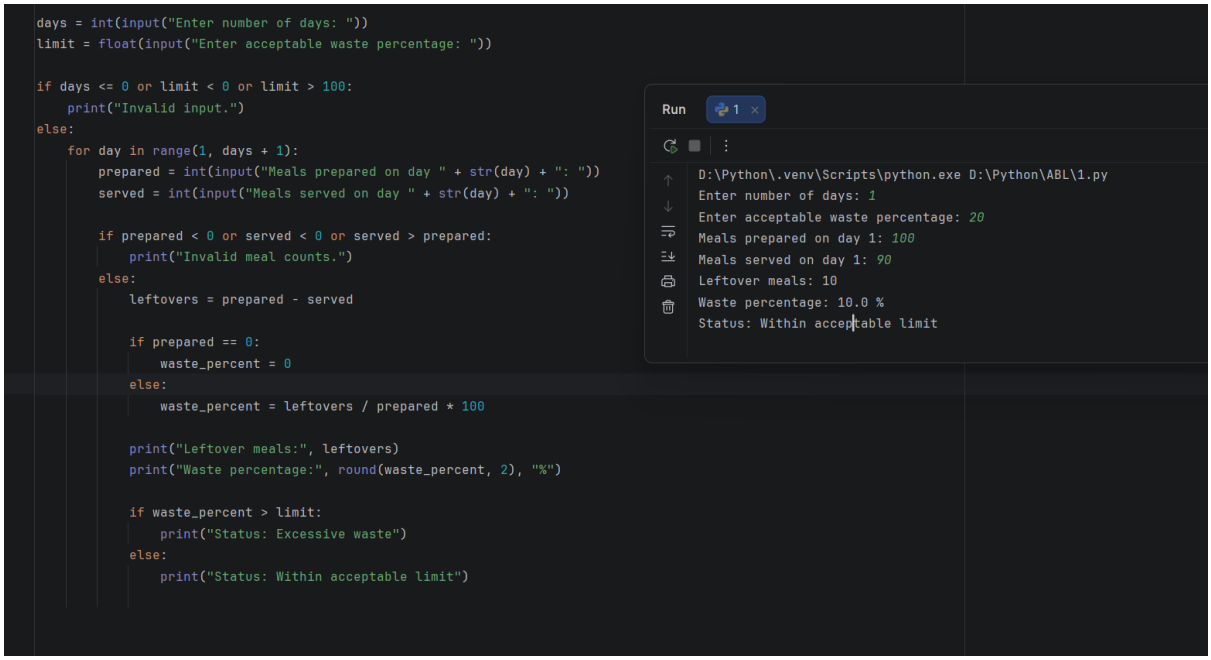
   Status: Within acceptable limit

2. Enter number of days: 1
   Enter acceptable waste percentage: 15
   Meals prepared on day 1: 200
   Meals served on day 1: 150
   Output:
   Leftover meals: 50

Waste percentage: 25.0 %
Status: Excessive waste

Output





Conclusion

The program uses arithmetic and conditions to monitor leftover food and flag days with high waste

3.

Problem statement

Calculate an electricity bill based on the number of units consumed, using different rates for different usage levels.

Objectives

- Read electricity units.
- Calculate the bill using progressive rates.
- Display the total bill.

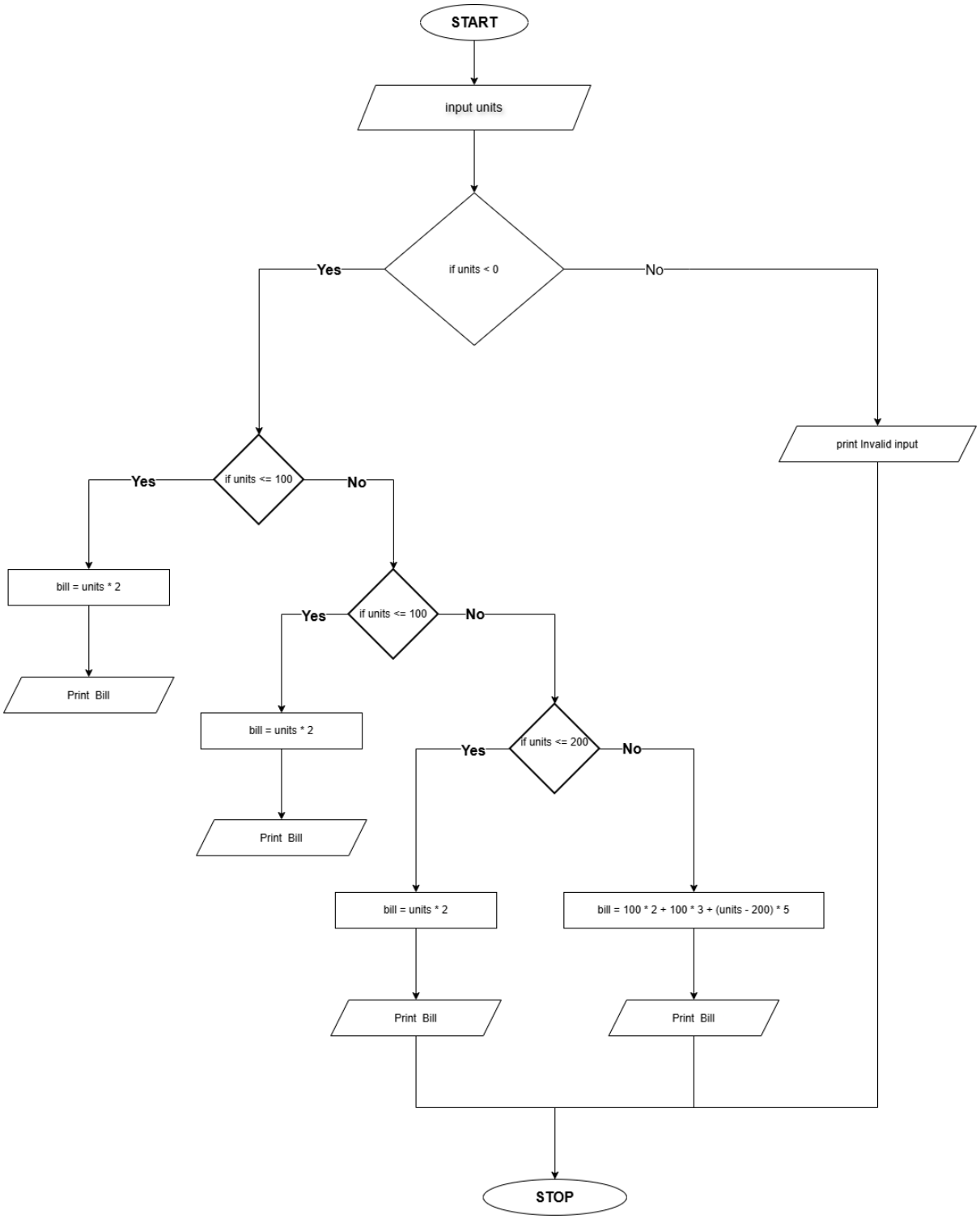
Assumptions

- First 100 units: ₹2 per unit.
- Next 100 units: ₹3 per unit.
- Units above 200: ₹5 per unit.
- No additional fixed charges.

Algorithm

1. Start.
2. Read units consumed.
3. If units are negative, display invalid input.
4. Otherwise, calculate the bill using the applicable slabs.
5. Display the bill.
6. Stop.

Flowchart



Pseudocode

Start

Input units

if units < 0

```
   print  "Invalid input"
else    IF units <= 100
      bill = units * 2
   ELSE IF units <= 200
      bill = 100 * 2 + (units - 100) * 3
   ELSE
      bill = 100 * 2 + 100 * 3 + (units - 200) * 5
   PRINT bill
STOP
```

Source Code

```
units = int(input("Enter electricity units: "))

if units < 0:
   print("Invalid input.")
elif units <= 100:
   bill = units * 2
   print("Electricity bill: Rs.", bill)
elif units <= 200:
   bill = 100 * 2 + (units - 100) * 3
   print("Electricity bill: Rs.", bill)
else:
   bill = 100 * 2 + 100 * 3 + (units - 200) * 5
   print("Electricity bill: Rs.", bill)
```

Test Cases

1. Enter electricity units: 80
   Electricity bill: Rs. 160
2. Enter electricity units: 250

   Electricity bill: Rs. 750

Output

```python
units = int(input("Enter electricity units: "))

if units < 0:
    print("Invalid input.")
elif units <= 100:
    bill = units * 2
    print("Electricity bill: Rs.", bill)
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
    print("Electricity bill: Rs.", bill)
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
    print("Electricity bill: Rs.", bill)
```

```
1 ×

⋮

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter electricity units: 80
Electricity bill: Rs. 160

Process finished with exit code 0
```



Conclusion

The program calculates electricity charges using progressive rates.

4.

Hospital Billing and Patient Classification: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

Sol:

Problem statement

Calculate a patient's hospital bill based on the number of days admitted and the daily room charge. Classify the bill amount.

Objectives

- To calculate the hospital bill based on admission days.
- To classify the bill into Low, Medium, or High.
- To display the total bill and its category.

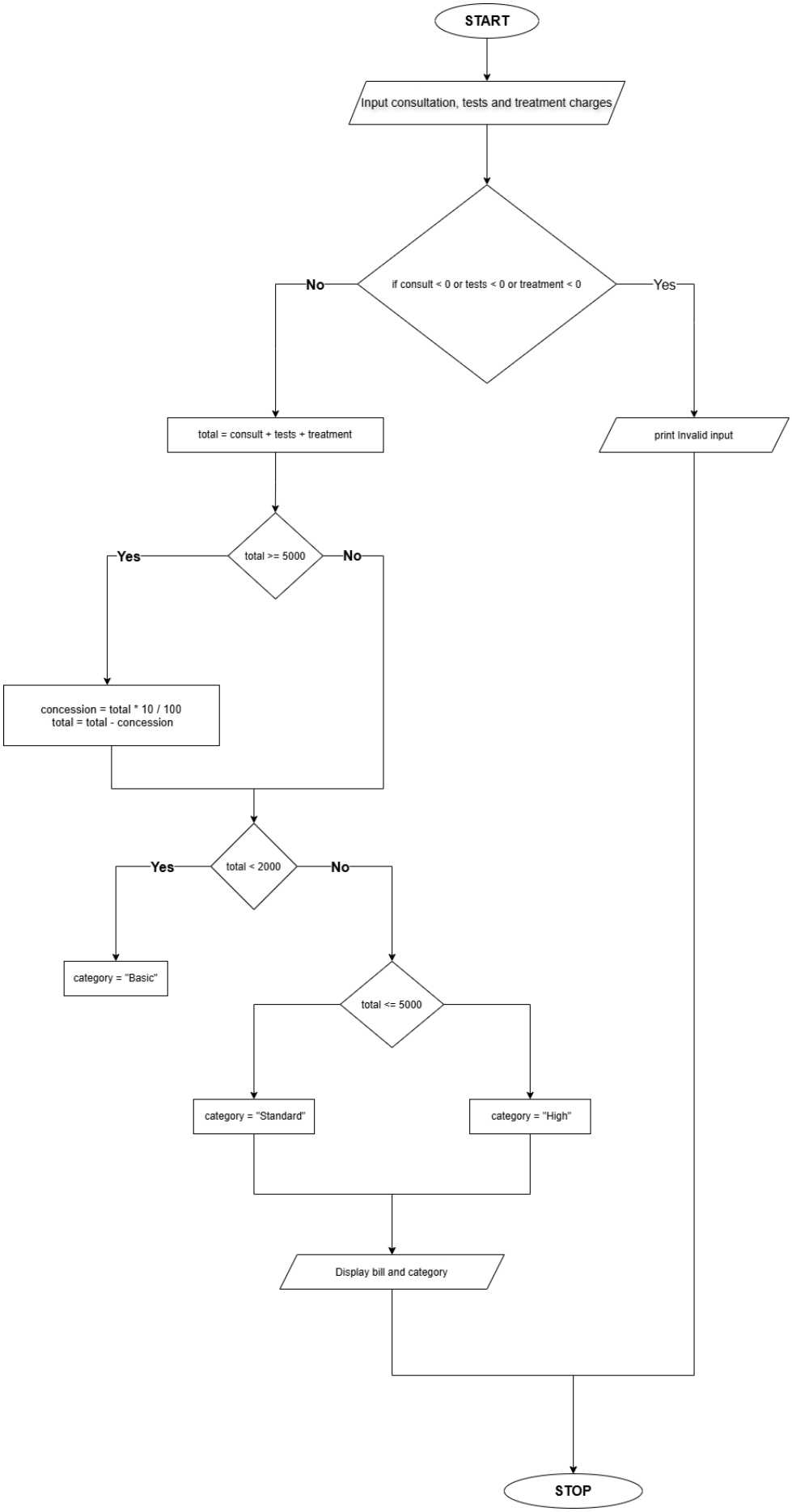
Assumptions

- Daily room charge is ₹1,000.
- Bills below ₹5,000 are classified as Low.
- Bills from ₹5,000 to ₹10,000 are Medium.
- Bills above ₹10,000 are High.

Algorithm

1. Read the number of admission days.
2. Validate the number of days.
3. Calculate the total bill.
4. Classify the bill amount.
5. Display the bill and classification.

Flowchart


START
Input consultation, tests and treatment charges
if consult < 0 or tests < 0 or treatment < 0
No
Yes
total = consult + tests + treatment
print Invalid input
total >= 5000
Yes
No
concession = total * 10 / 100
total = total - concession
total < 2000
Yes
No
category = "Basic"
total <= 5000
category = "Standard"
category = "High"
Display bill and category
STOP

Pseudocode

```
START
INPUT consultation, diagnostic, treatment
IF any charge is negative:
  DISPLAY "Invalid charges"
ELSE:
  total = consultation + diagnostic + treatment
  IF total >= 5000:
    total = total - total * 10 / 100
  IF total < 2000:
    category = "Basic"
  ELSE IF total <= 5000:
    category = "Standard"
  ELSE:
    category = "High"
  DISPLAY total
  DISPLAY category
STOP
```

Source Code

```
consult = float(input("Consultation charge: "))
tests = float(input("Diagnostic charges: "))
treatment = float(input("Treatment charges: "))
if consult < 0 or tests < 0 or treatment < 0:
  print("Invalid charges.")
else:
  total = consult + tests + treatment
  if total >= 5000:
```

```
    concession = total * 10 / 100
    total = total - concession
if total < 2000:
    category = "Basic"
elif total <= 5000:
    category = "Standard"
else:
    category = "High"
print("Final bill: Rs.", round(total, 2))
print("Patient category:", category)
```

Test Cases

1. Consultation charge: 500
   Diagnostic charges: 300
   Treatment charges: 700
   Output:
   Final bill: Rs. 1500.0
   Patient category: Basic
2. Consultation charge: 2000
   Diagnostic charges: 1500
   Treatment charges: 2500
   Output:
   Final bill: Rs. 5400.0
   Patient category: High

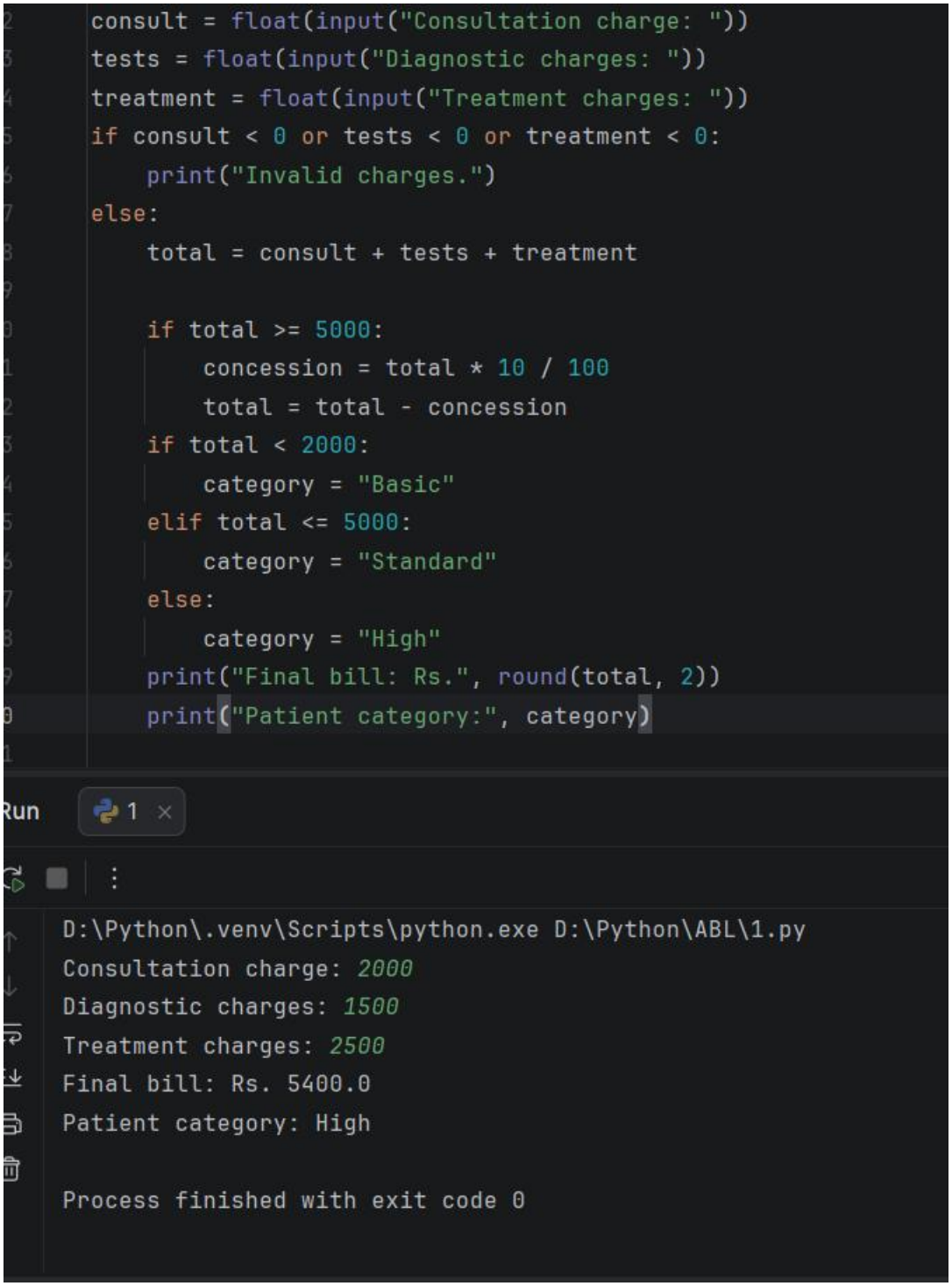
Output

```python
consult = float(input("Consultation charge: "))
tests = float(input("Diagnostic charges: "))
treatment = float(input("Treatment charges: "))
if consult < 0 or tests < 0 or treatment < 0:
    print("Invalid charges.")
else:
    total = consult + tests + treatment

    if total >= 5000:
        concession = total * 10 / 100
        total = total - concession
    if total < 2000:
        category = "Basic"
    elif total <= 5000:
        category = "Standard"
    else:
        category = "High"
    print("Final bill: Rs.", round(total, 2))
    print("Patient category:", category)
```

```
un  1 ×
:
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Consultation charge: 500
Diagnostic charges: 300
Treatment charges: 700
Final bill: Rs. 1500.0
Patient category: Basic

Process finished with exit code 0
```



Conclusion

The program calculates hospital bills, applies a concession when eligible, validates charges, and classifies patients using conditional statements.

5.

ATM Transaction Sequence Simulation: Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

Problem statement

Develop a Python program that processes a sequence of ATM deposits and withdrawals, updates the account balance, rejects invalid transactions, and displays a transaction summary.

Objectives

- Perform deposits and withdrawals.
- Maintain the current balance.
- Reject invalid or unaffordable transactions.
- Count successful and rejected transactions.

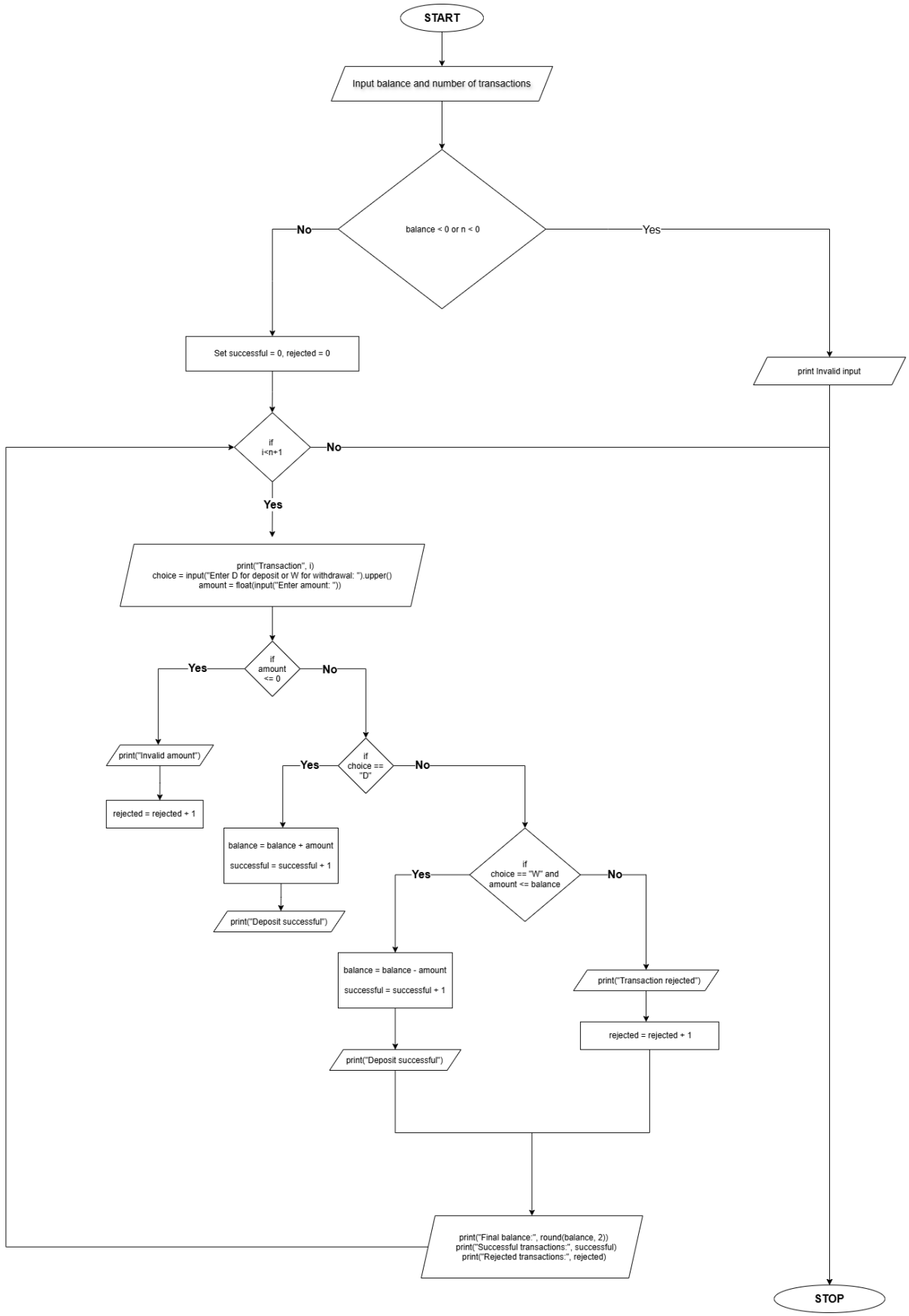
Assumptions

- The starting balance cannot be negative.
- The transaction type is D for deposit or W for withdrawal.
- Each transaction amount must be positive.
- Withdrawals cannot exceed the available balance.

Algorithm

1. Start.
2. Input the starting balance and number of transactions.
3. Set successful and rejected transaction counts to zero.
4. Repeat for each transaction.
5. Input the transaction type and amount.
6. If the transaction is valid, update the balance and successful count.
7. Otherwise, increase the rejected count.
8. Display the final balance and transaction counts.
9. Stop.

Flowchart


START
Input balance and number of transactions
balance < 0 or n < 0
No
Yes
print Invalid input
Set successful = 0, rejected = 0
if i<n+1
No
Yes
print("Transaction", i)
choice = input("Enter D for deposit or W for withdrawal: ").upper()
amount = float(input("Enter amount: "))
if amount <= 0
Yes
No
print("Invalid amount")
rejected = rejected + 1
if choice == "D"
Yes
No
balance = balance + amount
successful = successful + 1
print("Deposit successful")
if choice == "W" and amount <= balance
Yes
No
balance = balance - amount
successful = successful + 1
print("Deposit successful")
print("Transaction rejected")
rejected = rejected + 1
print("Final balance:", round(balance, 2))
print("Successful transactions:", successful)
print("Rejected transactions:", rejected)
STOP

Pseudocode

```
START
INPUT balance, n
SET successful = 0
SET rejected = 0
REPEAT n times
   INPUT type, amount
   IF amount <= 0
      rejected = rejected + 1
   ELSE IF type = "D"
      balance = balance + amount
      successful = successful + 1
   ELSE IF type = "W" AND amount <= balance
      balance = balance - amount
      successful = successful + 1
   ELSE
      rejected = rejected + 1
PRINT final balance
PRINT successful transactions
PRINT rejected transactions
END
```

Source Code

```
balance = float(input("Enter starting balance: "))
n = int(input("How many transactions? "))
if balance < 0 or n < 0:
   print("Invalid input.")
```

```
else:
    successful = 0
    rejected = 0
    for i in range(1, n + 1):
        print("Transaction", i)
        choice = input("Enter D for deposit or W for withdrawal: ").upper()
        amount = float(input("Enter amount: "))
        if amount <= 0:
            print("Invalid amount")
            rejected = rejected + 1
        elif choice == "D":
            balance = balance + amount
            successful = successful + 1
            print("Deposit successful")
        elif choice == "W" and amount <= balance:
            balance = balance - amount
            successful = successful + 1
            print("Withdrawal successful")
        else:
            print("Transaction rejected")
            rejected = rejected + 1
    print("Final balance:", round(balance, 2))
    print("Successful transactions:", successful)
    print("Rejected transactions:", rejected)
```

Test Cases

1. Enter starting balance: 1000
   How many transactions? 2
   Transaction 1
   Enter D for deposit or W for withdrawal: D

Enter amount: 500
Transaction 2
Enter D for deposit or W for withdrawal: W
Enter amount: 200
Output:
Transaction 1
Deposit successful
Transaction 2
Withdrawal successful
Final balance: 1300.0
Successful transactions: 2
Rejected transactions: 0

2. Enter starting balance: 500
   How many transactions? 2
   Transaction 1
   Enter D for deposit or W for withdrawal: W
   Enter amount: 700
   Transaction 2
   Enter D for deposit or W for withdrawal: D
   Enter amount: 200
   Output
   Transaction 1
   Transaction rejected
   Transaction 2
   Deposit successful
   Final balance: 700.0
   Successful transactions: 1
   Rejected transactions: 1

Output

```python
balance = float(input("Enter starting balance: "))
n = int(input("How many transactions? "))
if balance < 0 or n < 0:
    print("Invalid input.")
else:
    successful = 0
    rejected = 0
    for i in range(1, n + 1):
        print("Transaction", i)
        choice = input("Enter D for deposit or W for withdrawal: ").upper()
        amount = float(input("Enter amount: "))
        if amount <= 0:
            print("Invalid amount")
            rejected = rejected + 1
        elif choice == "D":
            balance = balance + amount
            successful = successful + 1
            print("Deposit successful")
        elif choice == "W" and amount <= balance:
            balance = balance - amount
            successful = successful + 1
            print("Withdrawal successful")
        else:
            print("Transaction rejected")
            rejected = rejected + 1
    print("Final balance:", round(balance, 2))
    print("Successful transactions:", successful)
    print("Rejected transactions:", rejected)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter starting balance: 500
How many transactions? 2
Transaction 1
Enter D for deposit or W for withdrawal: w
Enter amount: 700
Transaction rejected
Transaction 2
Enter D for deposit or W for withdrawal: d
Enter amount: 200
Deposit successful
Final balance: 700.0
Successful transactions: 1
Rejected transactions: 1

Process finished with exit code 0
```

```python
balance = float(input("Enter starting balance: "))
n = int(input("How many transactions? "))
if balance < 0 or n < 0:
    print("Invalid input.")
else:
    successful = 0
    rejected = 0
    for i in range(1, n + 1):
        print("Transaction", i)
        choice = input("Enter D for deposit or W for withdrawal: ").upper()
        amount = float(input("Enter amount: "))
        if amount <= 0:
            print("Invalid amount")
            rejected = rejected + 1
        elif choice == "D":
            balance = balance + amount
            successful = successful + 1
            print("Deposit successful")
        elif choice == "W" and amount <= balance:
            balance = balance - amount
            successful = successful + 1
            print("Withdrawal successful")
        else:
            print("Transaction rejected")
            rejected = rejected + 1
    print("Final balance:", round(balance, 2))
    print("Successful transactions:", successful)
    print("Rejected transactions:", rejected)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter starting balance: 1000
How many transactions? 2
Transaction 1
Enter D for deposit or W for withdrawal: d
Enter amount: 500
Deposit successful
Transaction 2
Enter D for deposit or W for withdrawal: w
Enter amount: 200
Withdrawal successful
Final balance: 1300.0
Successful transactions: 2
Rejected transactions: 0

Process finished with exit code 0
```

Conclusion

The program simulates ATM transactions using loops, conditions, and arithmetic operations. It also tracks successful and rejected transactions.

6.

The program simulates ATM transactions using loops, conditions, and arithmetic operations. It also tracks successful and rejected transactions.

Sol:

Problem statement

Calculate passenger fares using distance, travel class, age concessions, reservation fees and cancellation penalties.

Objectives

- Calculate distance-based fares.
- Apply class charges and age concessions.
- Calculate reservation and cancellation charges.

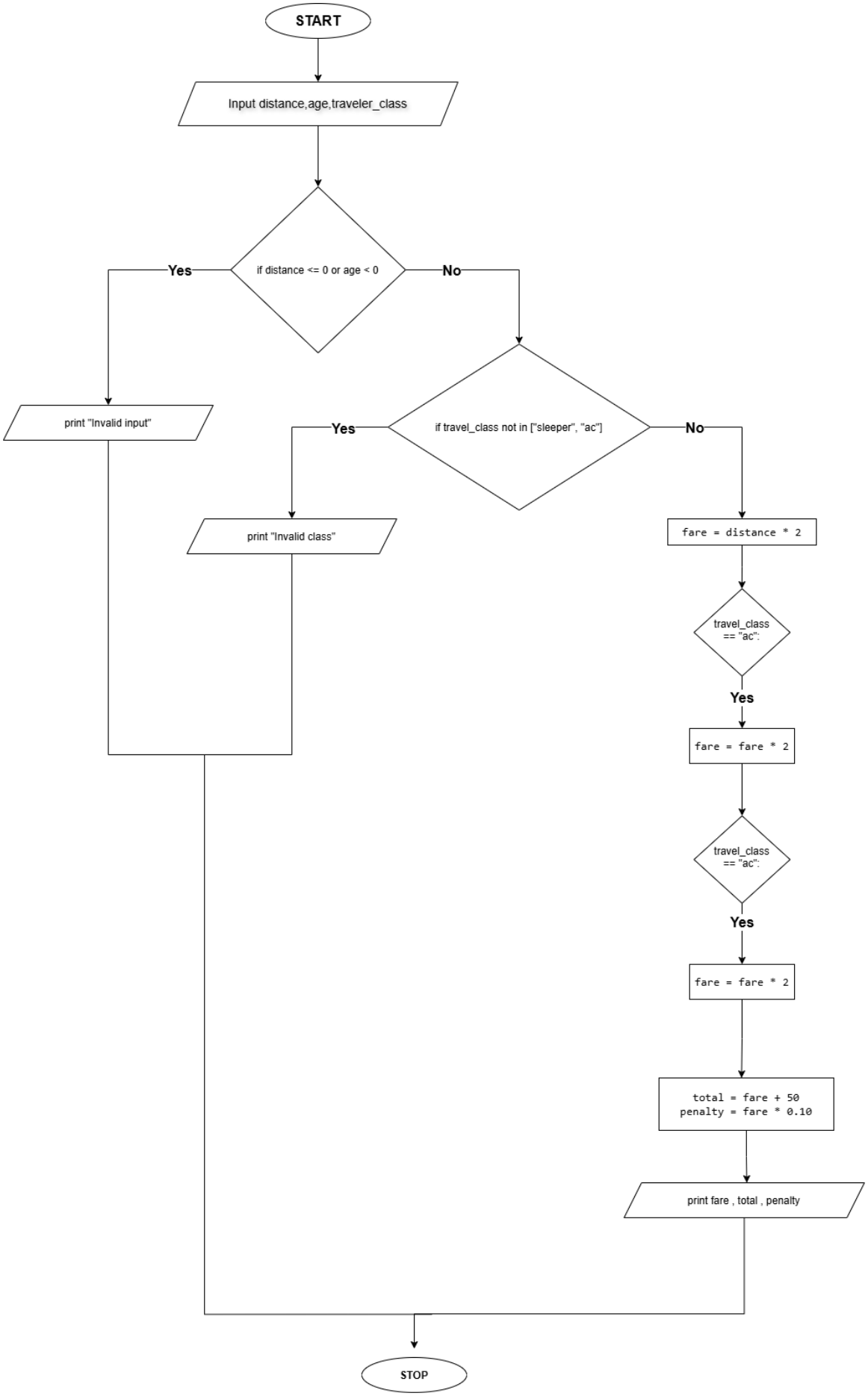
Assumptions

- Base fare is ₹2 per kilometre.
- Class multipliers are 1 for Sleeper and 2 for AC.
- Passengers aged 60 or above receive a 20% concession.
- Reservation fee is ₹50 per passenger.
- Cancellation penalty is 10% of the fare after concession.

Algorithm

1. Input passenger details, distance and class.
2. Calculate the base fare.
3. Apply the class multiplier.
4. Apply the age concession, if eligible.
5. Add the reservation fee.
6. Calculate the cancellation penalty.
7. Display the fare details.

Flowchart


START
Input distance,age,traveler_class
if distance <= 0 or age < 0
Yes
No
print "Invalid input"
if travel_class not in ["sleeper", "ac"]
Yes
No
print "Invalid class"
fare = distance * 2
travel_class == "ac":
Yes
fare = fare * 2
travel_class == "ac":
Yes
fare = fare * 2
total = fare + 50
penalty = fare * 0.10
print fare , total , penalty
STOP

Pseudocode

START

INPUT distance, age, class

CALCULATE base fare

APPLY class multiplier

IF age >= 60

  APPLY concession

END IF

ADD reservation fee

CALCULATE cancellation penalty

DISPLAY fare and penalty

STOP

Source Code

```
distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
travel_class = input("Enter Sleeper or AC: ").lower()
if distance <= 0 or age < 0:
  print("Invalid input")
elif travel_class not in ["sleeper", "ac"]:
  print("Invalid class")
else:
  fare = distance * 2
  if travel_class == "ac":
    fare = fare * 2
  if age >= 60:
    fare = fare * 0.80
  total = fare + 50
  penalty = fare * 0.10
  print("Fare after concession:", fare)
  print("Total fare:", total)
  print("Cancellation penalty:", penalty)
```

Test Cases

1. Enter distance in km: 200
   Enter passenger age: 25
   Enter Sleeper or AC: Sleeper
   Output
   Fare after concession: 400.0
   Total fare: 450.0
   Cancellation penalty: 40.0
2. Enter distance in km: 300
   Enter passenger age: 65
   Enter Sleeper or AC: AC
   Output
   Fare after concession: 960.0
   Total fare: 1010.0
   Cancellation penalty: 96.0

Output

```
distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
travel_class = input("Enter Sleeper or AC: ").lower()
if distance <= 0 or age < 0:
    print("Invalid input")
elif travel_class not in ["sleeper", "ac"]:
    print("Invalid class")
else:
    fare = distance * 2
    if travel_class == "ac":
        fare = fare * 2

    if age >= 60:
        fare = fare * 0.80
    total = fare + 50
    penalty = fare * 0.10
    print("Fare after concession:", fare)
    print("Total fare:", total)
    print("Cancellation penalty:", penalty)
```

Run 1

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter distance in km: 200
Enter passenger age: 25
Enter Sleeper or AC: sleeper
Fare after concession: 400.0
Total fare: 450.0
Cancellation penalty: 40.0

Process finished with exit code 0
```

```python
distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
travel_class = input("Enter Sleeper or AC: ").lower()
if distance <= 0 or age < 0:
    print("Invalid input")
elif travel_class not in ["sleeper", "ac"]:
    print("Invalid class")
else:
    fare = distance * 2
    if travel_class == "ac":
        fare = fare * 2

    if age >= 60:
        fare = fare * 0.80
    total = fare + 50
    penalty = fare * 0.10
    print("Fare after concession:", fare)
    print("Total fare:", total)
    print("Cancellation penalty:", penalty)

```

Run 1

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter distance in km: 300
Enter passenger age: 65
Enter Sleeper or AC: ac
Fare after concession: 960.0
Total fare: 1010.0
Cancellation penalty: 96.0

Process finished with exit code 0
```

Conclusion

The program calculates railway fares and cancellation penalties according to the stated rules.

7.

Smart Parking Fee Management: Calculate charges from entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

Sol:

Problem statement

Calculate parking fees using entry and exit times, vehicle categories, overnight parking, daily limits and penalties.

Objectives

- Calculate parking duration.
- Apply different vehicle rates.
- Apply daily limits and overnight charges.

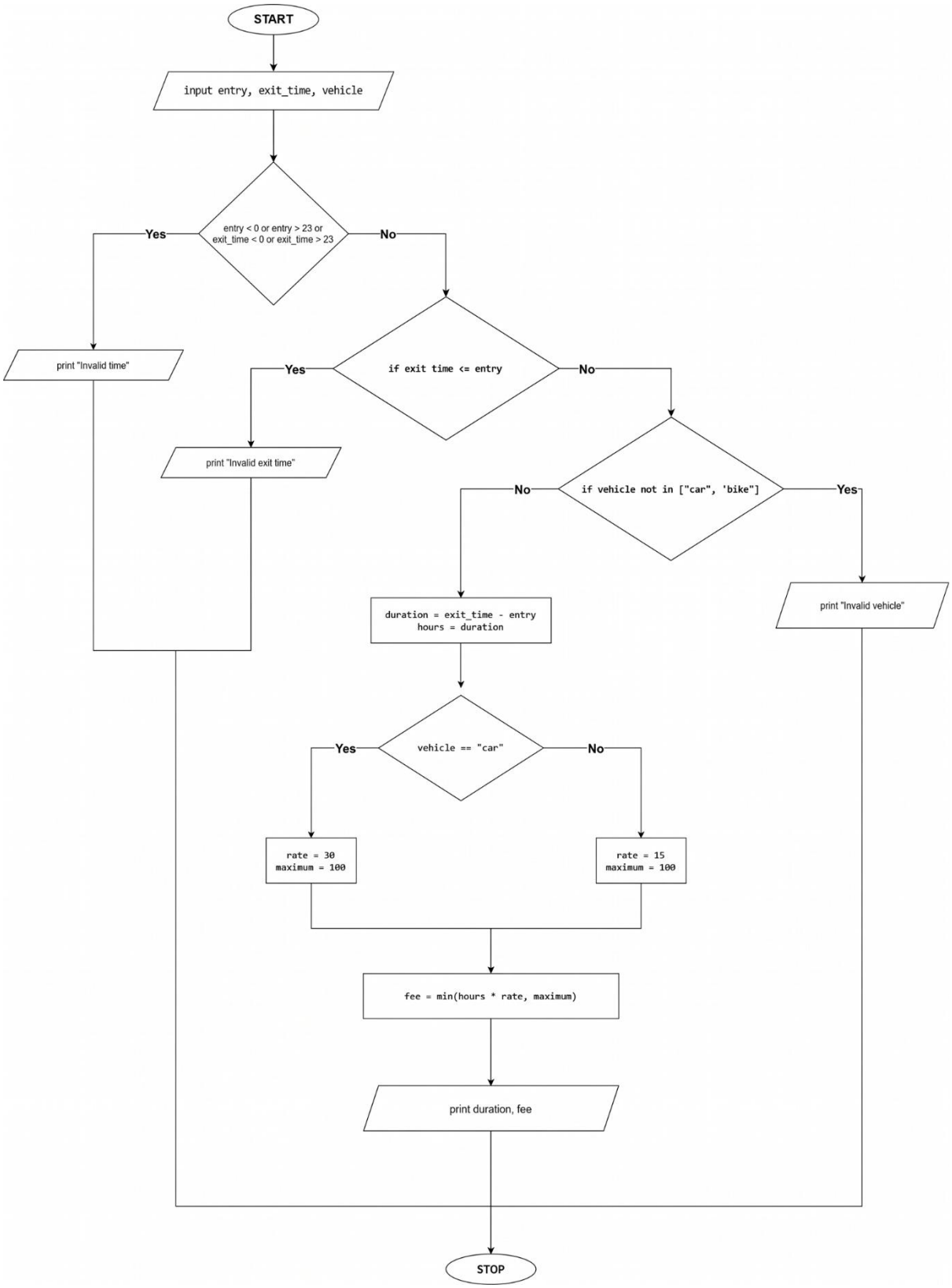
Assumptions

- Entry and exit are entered as hours from 0 to 23.
- Duration is calculated within one day; exit must be later than entry.
- Car rate is ₹30 per hour; bike rate is ₹15 per hour.
- Any partial hour is charged as a full hour.
- Maximum charge is ₹200 for a car and ₹100 for a bike.

Algorithm

1. Input entry time, exit time and vehicle type.
2. Validate the inputs.
3. Calculate duration.
4. Round duration up to the next whole hour.
5. Calculate the fee.
6. Apply the daily maximum.
7. Display the fee.

Flowchart



Pseudocode

START

INPUT entry, exit, vehicle

IF inputs are invalid

  DISPLAY error

ELSE

  duration = exit - entry

  CALCULATE rounded hours

  SELECT vehicle rate and maximum

  fee = minimum of calculated fee and maximum

  DISPLAY fee

END IF

STOP

Source Code

```
entry = int(input("Entry hour (0-23): "))
exit_time = int(input("Exit hour (0-23): "))
vehicle = input("Vehicle (car/bike): ").lower()

if entry < 0 or entry > 23 or exit_time < 0 or exit_time > 23:
  print("Invalid time")
elif exit_time <= entry:
  print("Invalid exit time")
elif vehicle not in ["car", "bike"]:
  print("Invalid vehicle")
else:
  duration = exit_time - entry
  hours = duration

  if vehicle == "car":
    rate = 30
    maximum = 200
  else:
    rate = 15
    maximum = 100
```

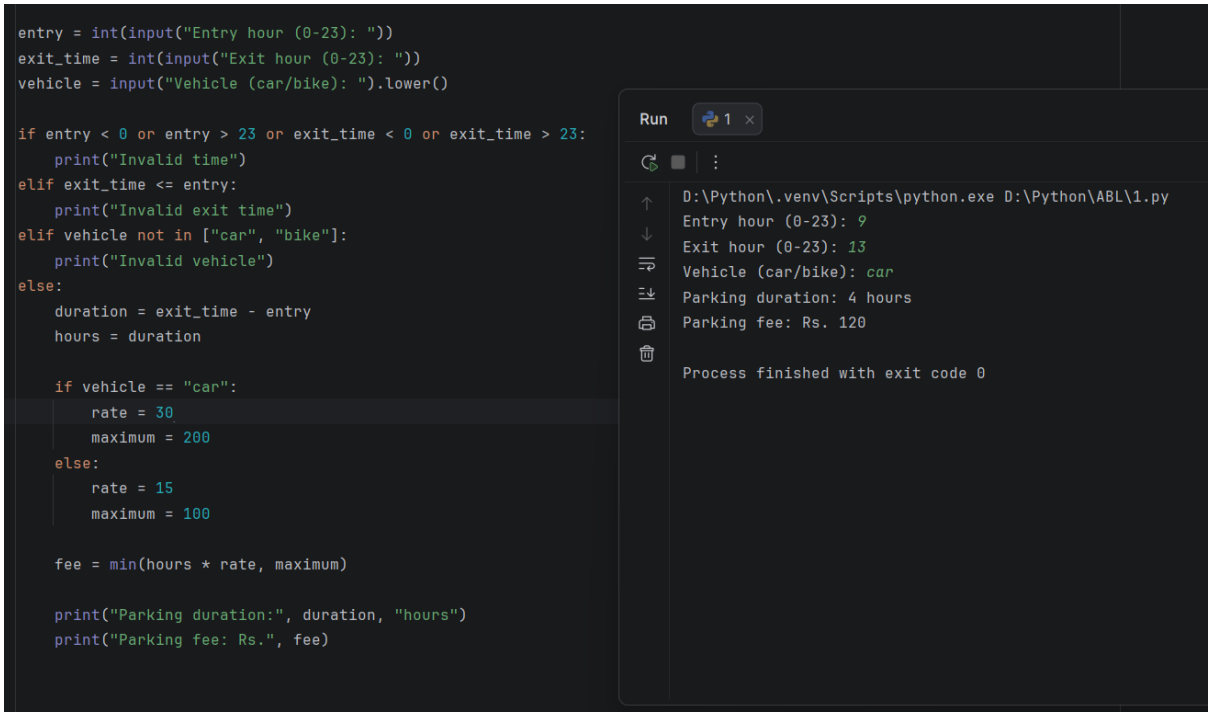
```
fee = min(hours * rate, maximum)

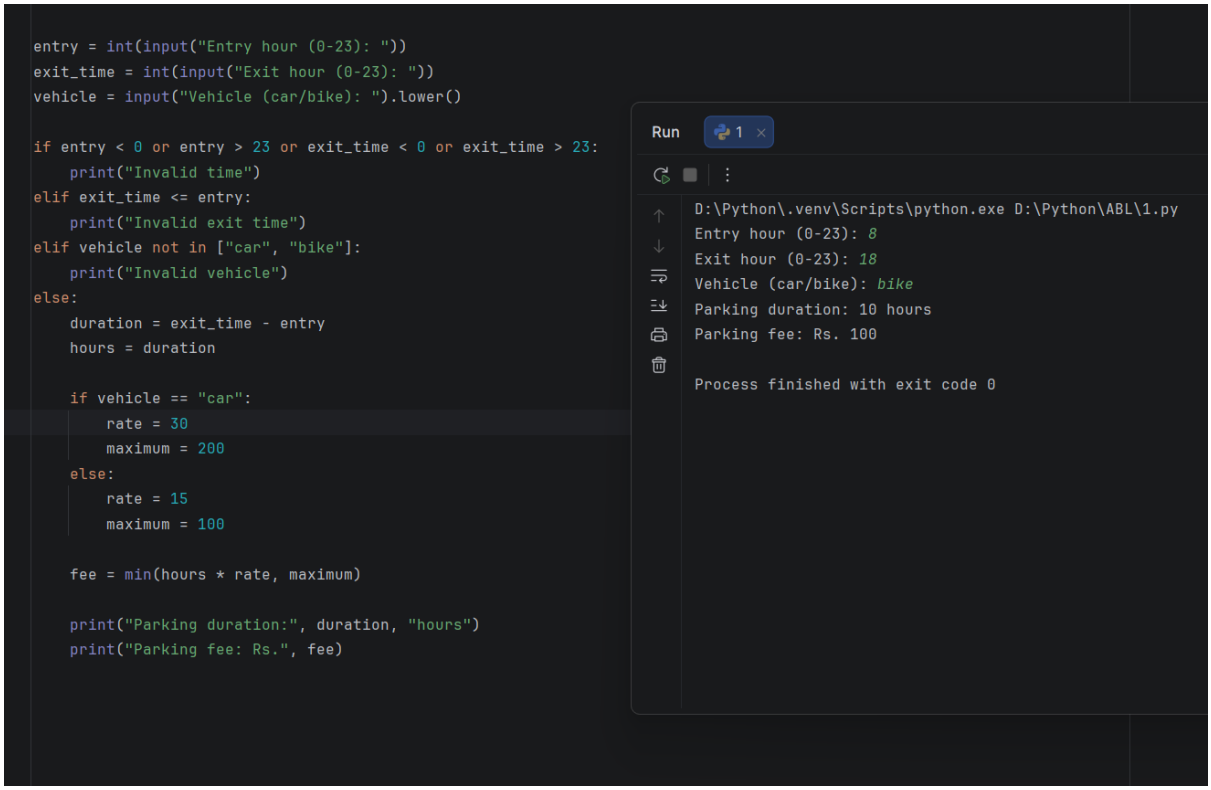
print("Parking duration:", duration, "hours")
print("Parking fee: Rs.", fee)
```

Test Cases

1. Entry hour (0-23): 9
   Exit hour (0-23): 13
   Vehicle (car/bike): car
   Output
   Parking duration: 4 hours
   Parking fee: Rs. 120
2. Entry hour (0-23): 8
   Exit hour (0-23): 18
   Vehicle (car/bike): bike
   Output
   Parking duration: 10 hours
   Parking fee: Rs. 100

Output





Conclusion

The program calculates parking fees using vehicle rates and maximum charges.

8.

Student Performance Trend Analysis: Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

Problem Statement

Compare assessment scores across examinations, calculate improvements and declines, identify weak subjects, and classify students.

Objectives

- Calculate score changes.
- Identify weak subjects.
- Classify performance.

Assumptions

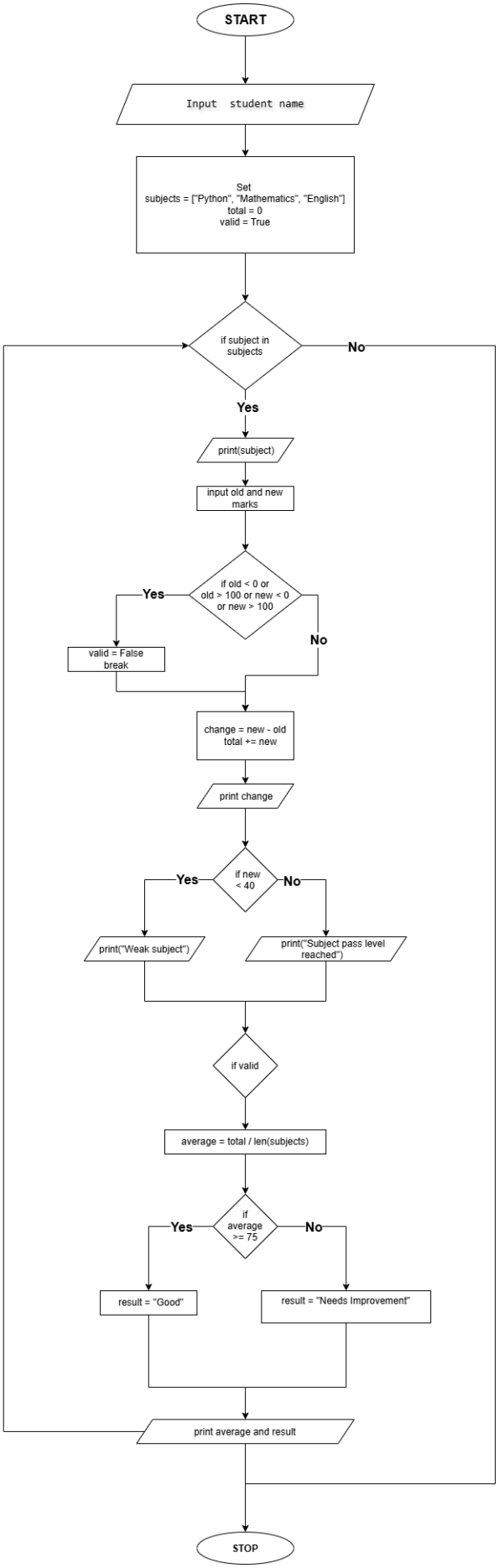
- Three subjects and two examinations are considered.
- Scores range from 0 to 100.

- A subject is weak if the latest score is below 40.
- Average score of 75 or more is classified as Good; otherwise, Needs Improvement.

Algorithm

1. Input the student's name.
2. Input earlier and latest scores for three subjects.
3. Calculate the average latest score.
4. Calculate each subject's score change.
5. Identify weak subjects.
6. Classify overall performance.
7. Display the report.

Flowchart


START
Input student name
Set
subjects = ["Python", "Mathematics", "English"]
total = 0
valid = True
if subject in subjects
No
Yes
print(subject)
input old and new marks
if old < 0 or old > 100 or new < 0 or new > 100
Yes
No
valid = False
break
change = new - old
total += new
print change
if new < 40
Yes
No
print("Weak subject")
print("Subject pass level reached")
if valid
average = total / len(subjects)
if average >= 75
Yes
No
result = "Good"
result = "Needs Improvement"
print average and result
STOP

Pseudocode

START

INPUT student name

FOR each subject

  INPUT earlier and latest marks

  CALCULATE change

  CHECK weak subject

END FOR

CALCULATE average

IF average >= 75

  classification = Good

ELSE

  classification = Needs Improvement

END IF

DISPLAY report

STOP

Source Code

```
name = input("Student name: ")
subjects = ["Python", "Mathematics", "English"]
total = 0
valid = True
for subject in subjects:
  print(subject)
  old = int(input("Earlier marks: "))
  new = int(input("Latest marks: "))
  if old < 0 or old > 100 or new < 0 or new > 100:
    valid = False
    print("Invalid marks")
    break

  change = new - old
```

```
    total += new
    print("Change:", change)
    if new < 40:
        print("Weak subject")
    else:
        print("Subject pass level reached")

if valid==True:
    average = total / len(subjects)
    if average >= 75:
        result = "Good"
    else:
        result = "Needs Improvement"

    print("Average:", average)
    print("Performance:", result)
```

Test Cases

1. Student name:Mikasa
   Python
   Earlier marks: 60
   Latest marks: 80
   Mathematics
   Earlier marks: 70
   Latest marks: 85
   English
   Earlier marks: 65
   Latest marks: 90
   Output
   Python
   Change: 20
   Subject pass level reached
   Mathematics
   Change: 15
   Subject pass level reached
   English
   Change: 25
   Subject pass level reached
   Average: 85.0
   Performance: Good

2. Student name:Shadow
   Python
   Earlier marks: 50

Latest marks: 35
Mathematics
Earlier marks: 60
Latest marks: 55
English
Earlier marks: 70
Latest marks: 60
Output
Python
Change: -15
Weak subject
Mathematics
Change: -5
Subject pass level reached
English
Change: -10
Subject pass level reached
Average: 50.0
Performance: Needs Improvement

Output

```python
total = 0
valid = True
for subject in subjects:
    print(subject)
    old = int(input("Earlier marks: "))
    new = int(input("Latest marks: "))
    if old < 0 or old > 100 or new < 0 or new > 100:
        valid = False
        print("Invalid marks")
        break

    change = new - old
    total += new
    print("Change:", change)
    if new < 40:
        print("Weak subject")
    else:
        print("Subject pass level reached")

if valid==True:
    average = total / len(subjects)
    if average >= 75:
        result = "Good"
    else:
        result = "Needs Improvement"

    print("Average:", average)
    print("Performance:", result)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Student name: Shadow
Python
Earlier marks: 50
Latest marks: 35
Change: -15
Weak subject
Mathematics
Earlier marks: 60
Latest marks: 55
Change: -5
Subject pass level reached
English
Earlier marks: 70
Latest marks: 60
Change: -10
Subject pass level reached
Average: 50.0
Performance: Needs Improvement

Process finished with exit code 0
```

```python
total = 0
valid = True
for subject in subjects:
    print(subject)
    old = int(input("Earlier marks: "))
    new = int(input("Latest marks: "))
    if old < 0 or old > 100 or new < 0 or new > 100:
        valid = False
        print("Invalid marks")
        break

    change = new - old
    total += new
    print("Change:", change)
    if new < 40:
        print("Weak subject")
    else:
        print("Subject pass level reached")

if valid==True:
    average = total / len(subjects)
    if average >= 75:
        result = "Good"
    else:
        result = "Needs Improvement"

    print("Average:", average)
    print("Performance:", result)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Student name: Mikasa
Python
Earlier marks: 60
Latest marks: 80
Change: 20
Subject pass level reached
Mathematics
Earlier marks: 70
Latest marks: 85
Change: 15
Subject pass level reached
English
Earlier marks:  65
Latest marks: 90
Change: 25
Subject pass level reached
Average: 85.0
Performance: Good

Process finished with exit code 0
```

Conclusion

The program analyses student performance across examinations and identifies improvements, declines and weak subjects.

9.

Courier Charge Optimization: Calculate delivery charges based on parcel weight, dimensions, distance zones, urgency, and declared restrictions; compare service options and select an eligible one.

Problem Statement

Calculate courier charges based on parcel weight, dimensions, distance zones, urgency, and restrictions. Compare service options and select an eligible one.

Objectives

- Calculate charges for different courier services.
- Apply urgency charges.
- Reject parcels exceeding service limits.
- Select the cheapest eligible service.

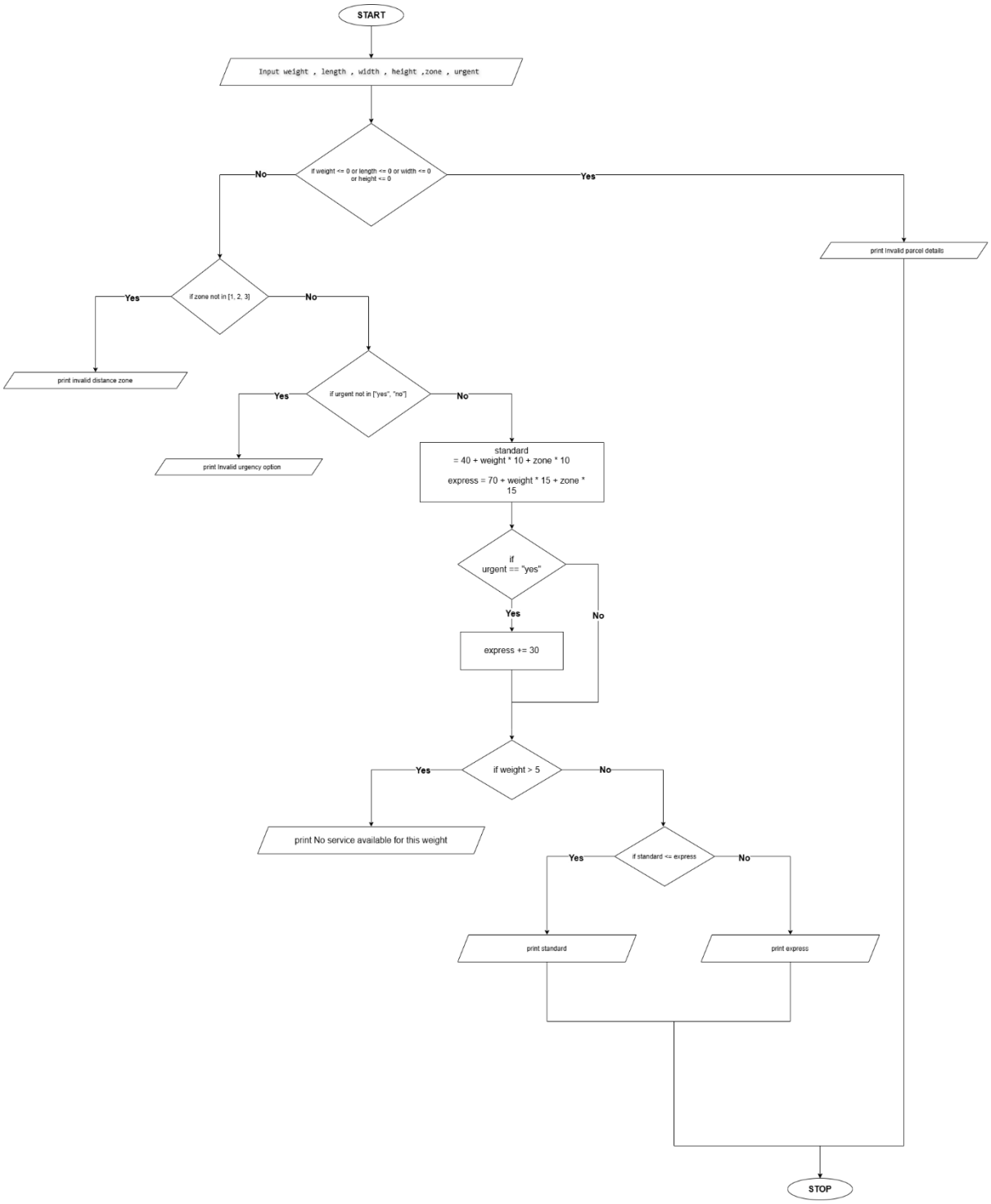
Assumptions

- Standard service costs ₹40 plus ₹10 per kg.
- Express service costs ₹70 plus ₹15 per kg.
- Parcels over 5 kg are not accepted.
- Express delivery adds ₹30 for urgent service.
- The parcel must have positive weight and valid dimensions.

Algorithm

1. Input parcel weight, dimensions, distance zone and urgency.
2. Validate the inputs.
3. Calculate the standard and express charges.
4. Check each service's eligibility.
5. Compare eligible charges.
6. Display the cheapest eligible service.

Flowchart



Pseudocode

START

INPUT weight, dimensions, zone, urgency

VALIDATE inputs

CALCULATE standard and express charges

CHECK service eligibility

SELECT cheapest eligible service

DISPLAY result

STOP

Source Code

```
weight = float(input("Parcel weight in kg: "))
length = float(input("Length in cm: "))
width = float(input("Width in cm: "))
height = float(input("Height in cm: "))
zone = int(input("Distance zone (1-3): "))
urgent = input("Urgent delivery? (yes/no): ").lower()

if weight <= 0 or length <= 0 or width <= 0 or height <= 0:
  print("Invalid parcel details")
elif zone not in [1, 2, 3]:
  print("Invalid distance zone")
elif urgent not in ["yes", "no"]:
  print("Invalid urgency option")
else:
  standard = 40 + weight * 10 + zone * 10
  express = 70 + weight * 15 + zone * 15

  if urgent == "yes":
    express += 30

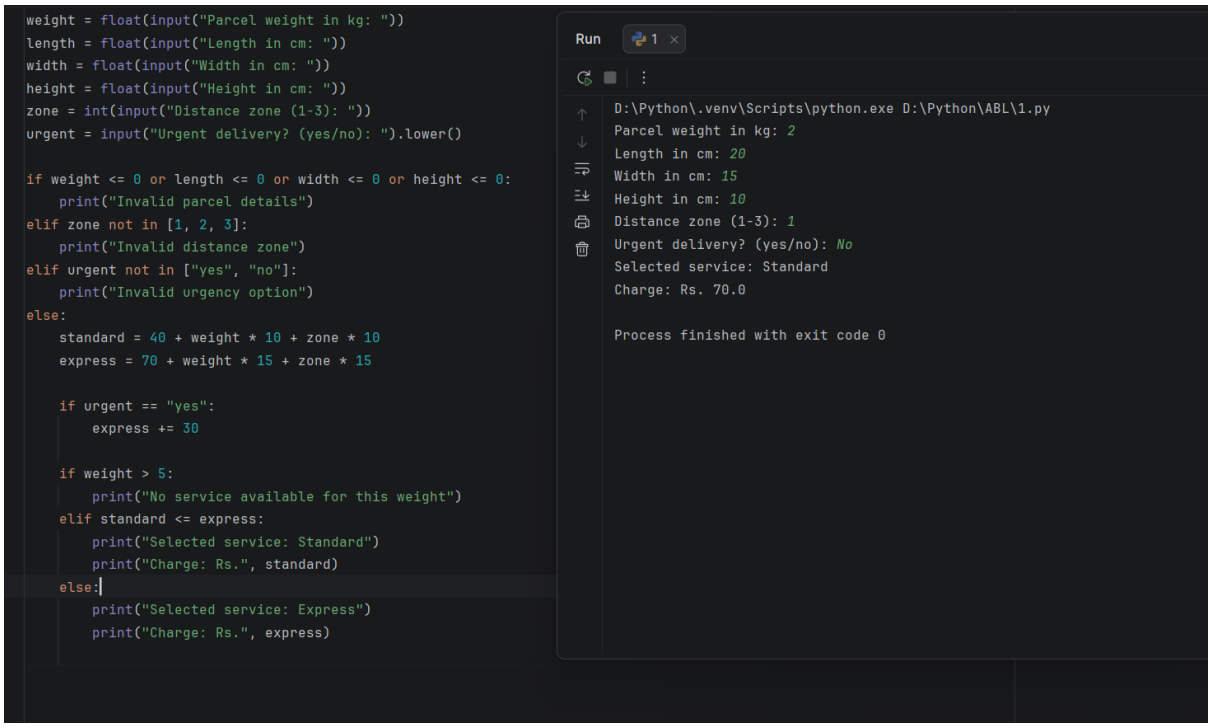
  if weight > 5:
    print("No service available for this weight")
  elif standard <= express:
    print("Selected service: Standard")
    print("Charge: Rs.", standard)
  else:
    print("Selected service: Express")
    print("Charge: Rs.", express)
```

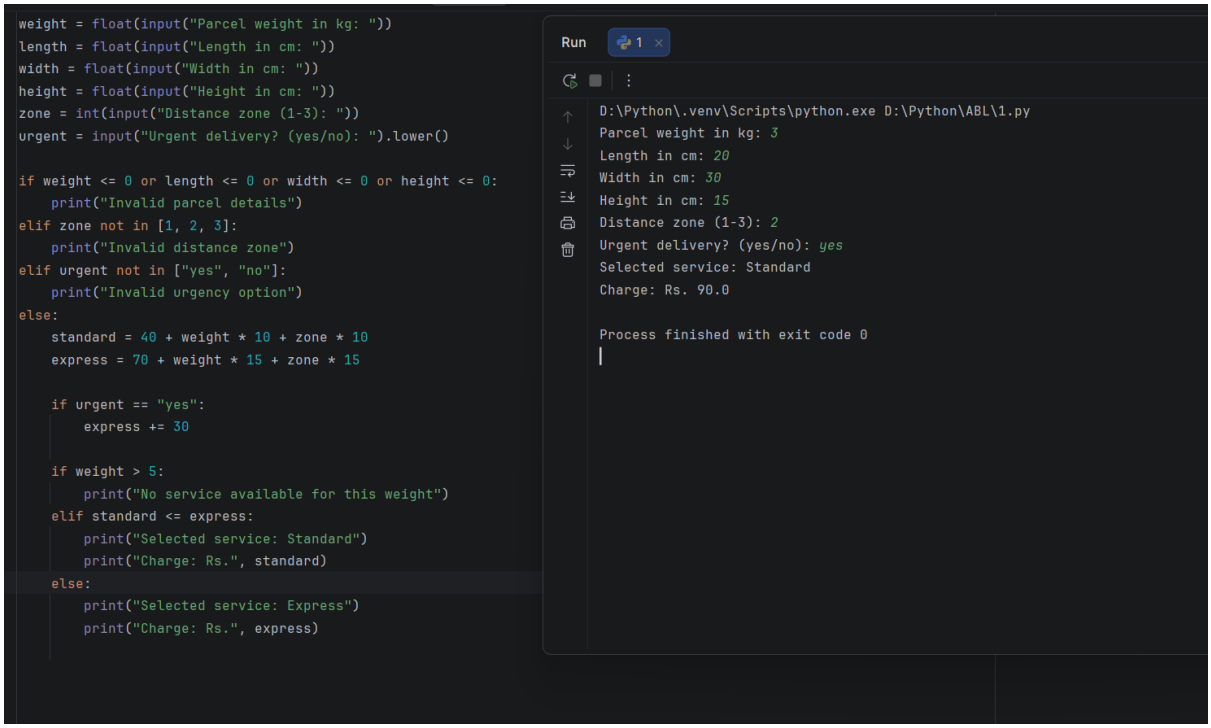
Test Cases

1. Parcel weight in kg: 2
   Length in cm: 20
   Width in cm: 15
   Height in cm: 10
   Distance zone (1-3): 1
   Urgent delivery? (yes/no): no
   Output
   Selected service: Standard
   Charge: Rs. 70.0

2. Parcel weight in kg: 3
   Length in cm: 30
   Width in cm: 20
   Height in cm: 15
   Distance zone (1-3): 2
   Urgent delivery? (yes/no): yes
   Output
   Selected service: Standard
   Charge: Rs. 90.0

Output





```python
weight = float(input("Parcel weight in kg: "))
length = float(input("Length in cm: "))
width = float(input("Width in cm: "))
height = float(input("Height in cm: "))
zone = int(input("Distance zone (1-3): "))
urgent = input("Urgent delivery? (yes/no): ").lower()

if weight <= 0 or length <= 0 or width <= 0 or height <= 0:
    print("Invalid parcel details")
elif zone not in [1, 2, 3]:
    print("Invalid distance zone")
elif urgent not in ["yes", "no"]:
    print("Invalid urgency option")
else:
    standard = 40 + weight * 10 + zone * 10
    express = 70 + weight * 15 + zone * 15

    if urgent == "yes":
        express += 30

    if weight > 5:
        print("No service available for this weight")
    elif standard <= express:
        print("Selected service: Standard")
        print("Charge: Rs.", standard)
    else:
        print("Selected service: Express")
        print("Charge: Rs.", express)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Parcel weight in kg: 3
Length in cm: 20
Width in cm: 30
Height in cm: 15
Distance zone (1-3): 2
Urgent delivery? (yes/no): yes
Selected service: Standard
Charge: Rs. 90.0

Process finished with exit code 0
```

Conclusion

The program compares courier services, calculates charges and selects the cheapest eligible option.

10.

Campus Transport Allocation: Determine the number of buses needed for several routes, account for bus capacities and passenger demand, and identify unserved passengers.

Problem statement

Develop a Python program to calculate the number of buses required for campus routes based on passenger demand and bus capacity. Identify the number of passengers who cannot be served if the available buses are insufficient.

Objectives

- Input passenger demand and bus capacity.
- Calculate the number of buses required.
- Allocate the available buses.
- Find unserved passengers.

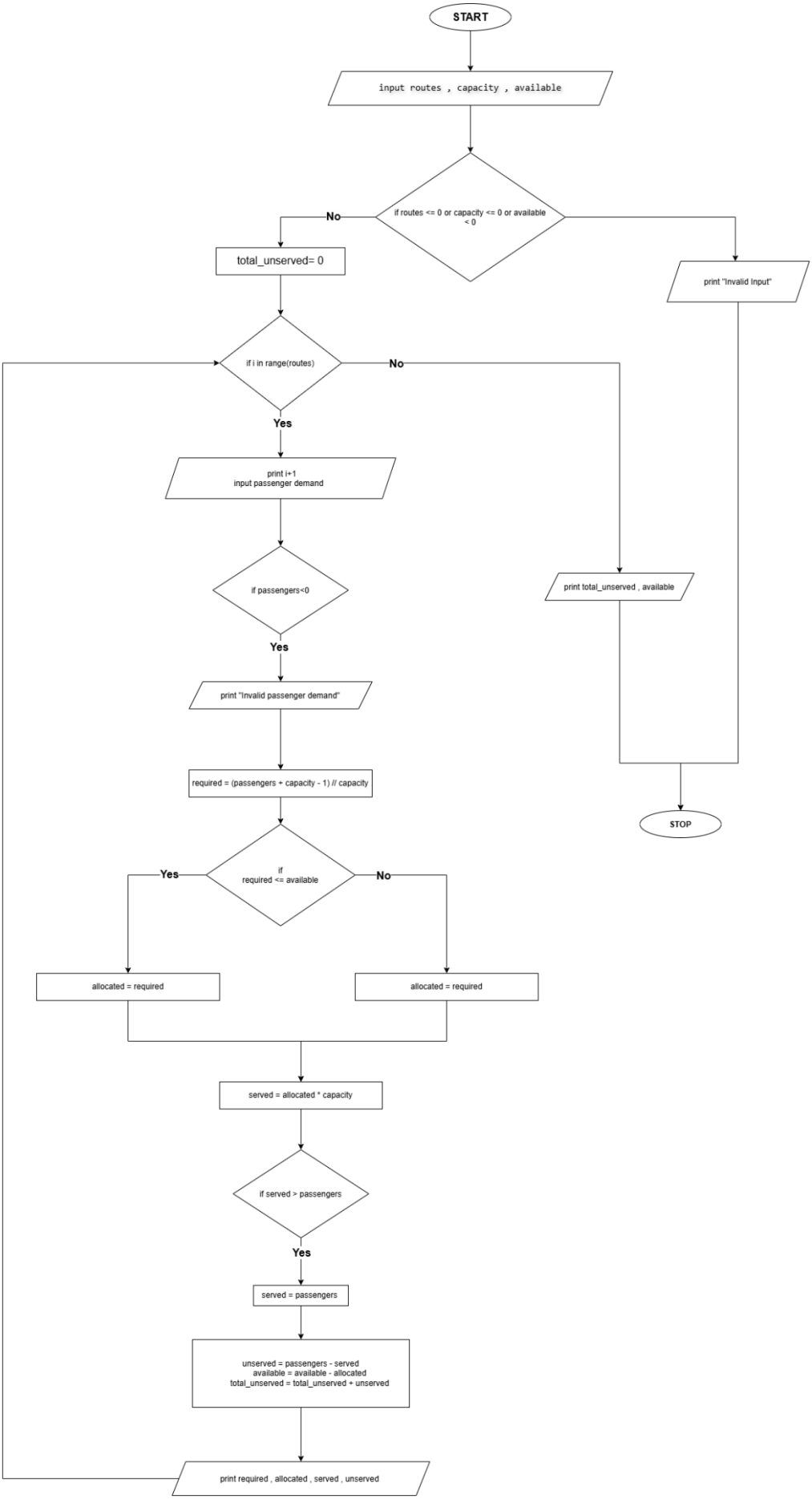
Assumptions

- Each bus has the same capacity.
- Each passenger requires one seat.
- The available bus count is shared across routes in input order.
- Passenger demand and bus capacity must be valid non-negative integers, with bus capacity greater than zero.

Algorithm

1. Start.
2. Input the number of routes.
3. Input the capacity of each bus and the number of available buses.
4. For each route, input passenger demand.
5. Calculate required buses using ceiling division.
6. Allocate available buses.
7. Calculate passengers served and unserved.
8. Display the result for each route.
9. Stop.

Flowchart


START
input routes , capacity , available
if routes <= 0 or capacity <= 0 or available < 0
No
print "Invalid Input"
total_unserved= 0
if i in range(routes)
No
Yes
print i+1
input passenger demand
print total_unserved , available
if passengers<0
Yes
print "Invalid passenger demand"
required = (passengers + capacity - 1) // capacity
STOP
if
required <= available
Yes
No
allocated = required
allocated = required
served = allocated * capacity
if served > passengers
Yes
served = passengers
unserved = passengers - served
available = available - allocated
total_unserved = total_unserved + unserved
print required , allocated , served , unserved

Pseudocode

START

INPUT number of routes

INPUT bus capacity

INPUT available buses

FOR each route

  INPUT passenger demand

  CALCULATE buses required

  ALLOCATE the smaller of:

    buses required

    remaining buses

  CALCULATE passengers served

  CALCULATE unserved passengers

  UPDATE remaining buses

  DISPLAY route summary

END FOR

STOP

Source Code

```
routes = int(input("Enter number of routes: "))
capacity = int(input("Enter capacity of each bus: "))
available = int(input("Enter available buses: "))

if routes <= 0 or capacity <= 0 or available < 0:
  print("Invalid input")
```

```
else:
    total_unserved = 0

    for i in range(routes):
        print("\nRoute", i + 1)
        passengers = int(input("Enter passenger demand: "))

        if passengers < 0:
            print("Invalid passenger demand")
            continue

        required = (passengers + capacity - 1) // capacity

        if required <= available:
            allocated = required
        else:
            allocated = available

        served = allocated * capacity

        if served > passengers:
            served = passengers

        unserved = passengers - served
        available = available - allocated
        total_unserved = total_unserved + unserved

        print("Buses required:", required)
        print("Buses allocated:", allocated)
        print("Passengers served:", served)
        print("Unserved passengers:", unserved)

    print("\nTotal unserved passengers:", total_unserved)
    print("Buses remaining:", available)
```

Test Cases

1. Enter number of routes: 2
   Enter capacity of each bus: 40

Enter available buses: 5

Route 1
Enter passenger demand: 75

Route 2
Enter passenger demand: 60
Output
Route 1
Buses required: 2
Buses allocated: 2
Passengers served: 75
Unserved passengers: 0

Route 2
Buses required: 2
Buses allocated: 2
Passengers served: 60
Unserved passengers: 0

Total unserved passengers: 0
Buses remaining: 1

2. Enter number of routes: 2
   Enter capacity of each bus: 30
   Enter available buses: 2

   Route 1
   Enter passenger demand: 50

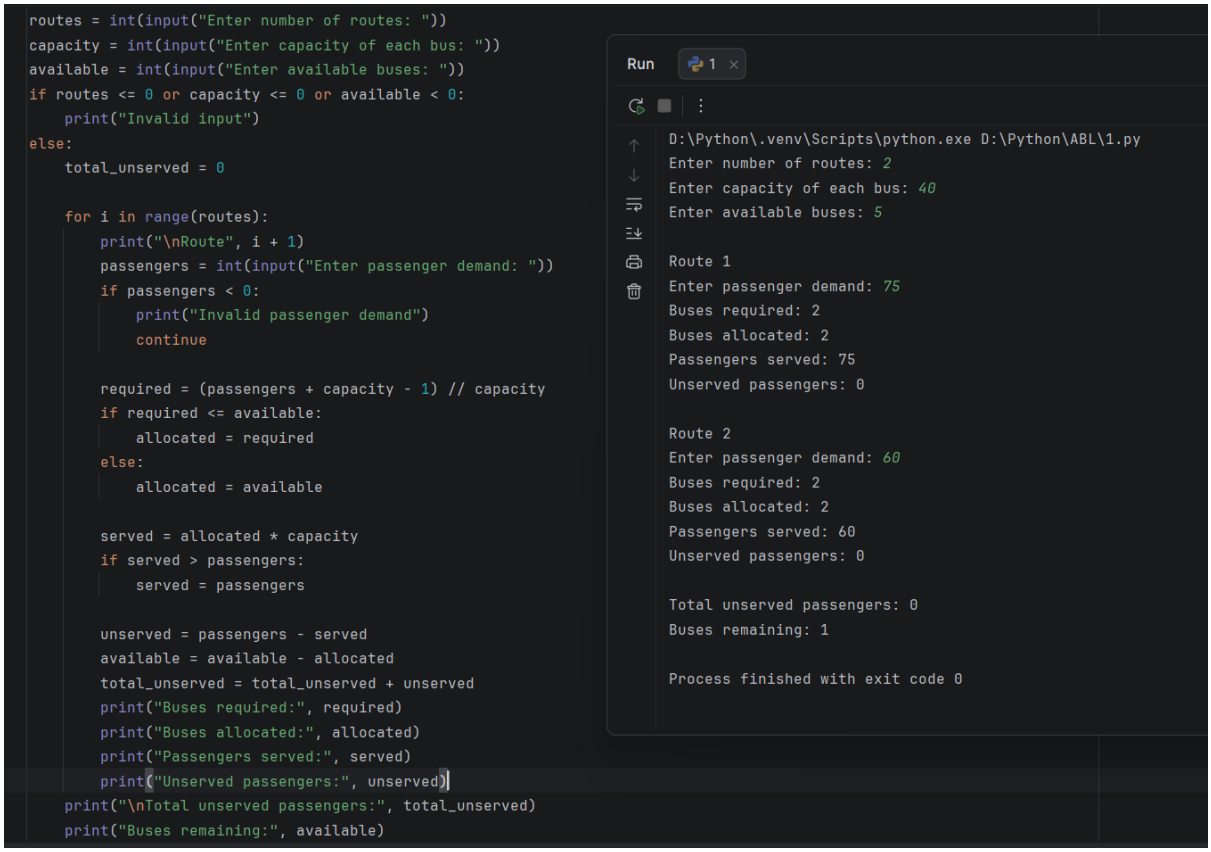
   Route 2
   Enter passenger demand: 40
   Output
   Route 1
   Buses required: 2
   Buses allocated: 2
   Passengers served: 50
   Unserved passengers: 0

   Route 2
   Buses required: 2

Buses allocated: 0
Passengers served: 0
Unserved passengers: 40

Total unserved passengers: 40
Buses remaining: 0

Output



```
routes = int(input("Enter number of routes: "))
capacity = int(input("Enter capacity of each bus: "))
available = int(input("Enter available buses: "))
if routes <= 0 or capacity <= 0 or available < 0:
    print("Invalid input")
else:
    total_unserved = 0

    for i in range(routes):
        print("\nRoute", i + 1)
        passengers = int(input("Enter passenger demand: "))
        if passengers < 0:
            print("Invalid passenger demand")
            continue

        required = (passengers + capacity - 1) // capacity
        if required <= available:
            allocated = required
        else:
            allocated = available

        served = allocated * capacity
        if served > passengers:
            served = passengers

        unserved = passengers - served
        available = available - allocated
        total_unserved = total_unserved + unserved
        print("Buses required:", required)
        print("Buses allocated:", allocated)
        print("Passengers served:", served)
        print("Unserved passengers:", unserved)
    print("\nTotal unserved passengers:", total_unserved)
    print("Buses remaining:", available)
```

```
Run  1

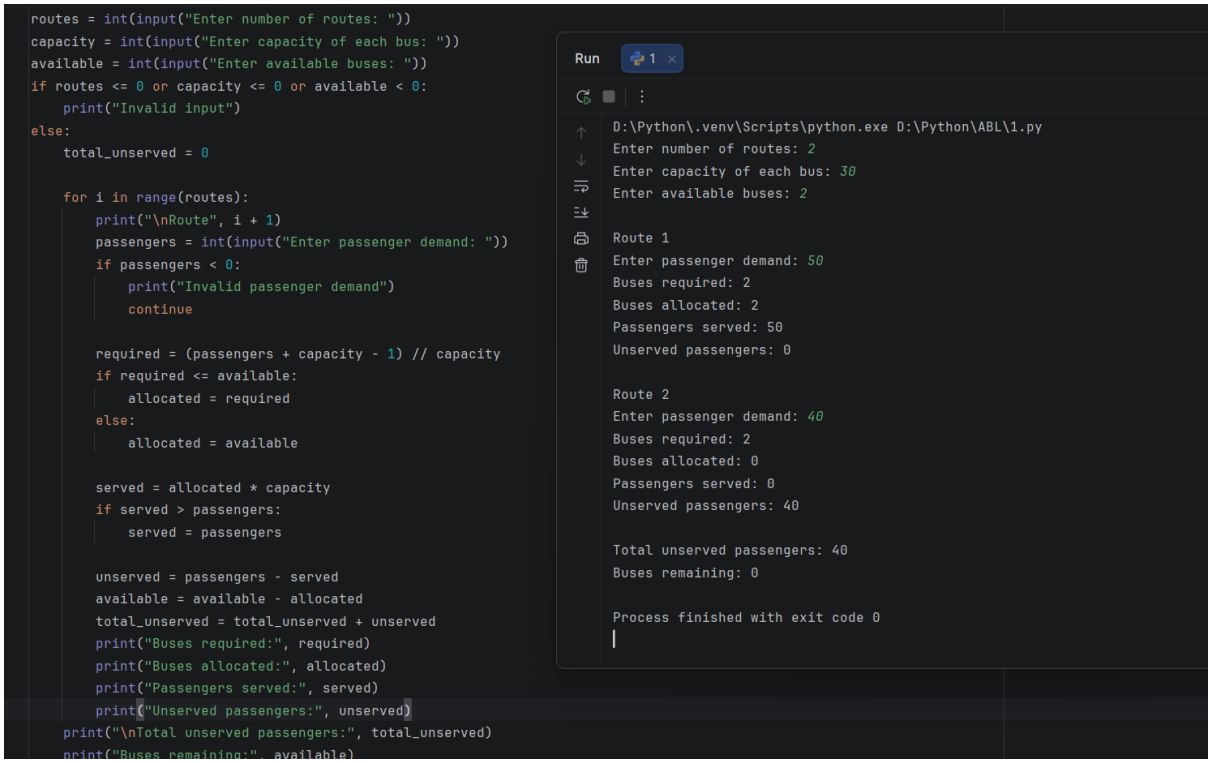
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of routes: 2
Enter capacity of each bus: 40
Enter available buses: 5

Route 1
Enter passenger demand: 75
Buses required: 2
Buses allocated: 2
Passengers served: 75
Unserved passengers: 0

Route 2
Enter passenger demand: 60
Buses required: 2
Buses allocated: 2
Passengers served: 60
Unserved passengers: 0

Total unserved passengers: 0
Buses remaining: 1

Process finished with exit code 0
```



```python
routes = int(input("Enter number of routes: "))
capacity = int(input("Enter capacity of each bus: "))
available = int(input("Enter available buses: "))
if routes <= 0 or capacity <= 0 or available < 0:
    print("Invalid input")
else:
    total_unserved = 0

    for i in range(routes):
        print("\nRoute", i + 1)
        passengers = int(input("Enter passenger demand: "))
        if passengers < 0:
            print("Invalid passenger demand")
            continue

        required = (passengers + capacity - 1) // capacity
        if required <= available:
            allocated = required
        else:
            allocated = available

        served = allocated * capacity
        if served > passengers:
            served = passengers

        unserved = passengers - served
        available = available - allocated
        total_unserved = total_unserved + unserved
        print("Buses required:", required)
        print("Buses allocated:", allocated)
        print("Passengers served:", served)
        print("Unserved passengers:", unserved)
    print("\nTotal unserved passengers:", total_unserved)
    print("Buses remaining:", available)
```

```
Run  1

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of routes: 2
Enter capacity of each bus: 30
Enter available buses: 2

Route 1
Enter passenger demand: 50
Buses required: 2
Buses allocated: 2
Passengers served: 50
Unserved passengers: 0

Route 2
Enter passenger demand: 40
Buses required: 2
Buses allocated: 0
Passengers served: 0
Unserved passengers: 40

Total unserved passengers: 40
Buses remaining: 0

Process finished with exit code 0
```

Conclusion

The program calculates bus requirements, allocates available buses in route order, and identifies passengers who cannot be served.